

CH06 ORM 框架和 MyBatis 详解

考试复习要点

1 ORM 框架概述

1.1 持久化与 ORM 的基本概念 [p.4]

- ▷ **持久化 (Persistence)**: 把瞬态数据 (如内存中的对象) 保存到可永久保存的存储设备中 (如磁盘文件、数据库等) [p.4]
- ▷ **持久化目标的三种类型** [p.4]:
 - **无结构文本文件**: 通过 I/O 技术读写文件 (如.txt)
 - **结构化的文本文件**: 通过 SDK 提供的 API 读写文件 (如.xml, .json, .xlsx)
 - **关系型数据库 (RDBMS)**: 通过数据库驱动技术 (如 JDBC) 读写 DBMS
- ▷ **ORM (Object-Relation Mapping, 对象关系映射)**: 完成瞬态的对象数据到持久的关系型数据映射的机制, 简称 ORM [p.4]
- ▷ **核心思想**: 将数据库表 (Table) 映射为 Java 类 (Class), 将表的行 (Row) 映射为 Java 对象 (Object), 将表的列 (Column) 映射为 Java 属性 (Field) [p.4]

1.2 为什么需要 ORM 框架? [p.5]

- ▷ **核心痛点**: 在 Java 应用中, 数据库表的结构与 Java 对象的结构不一致, 开发者不得不编写大量重复的 JDBC 代码来执行 SQL、设置参数、处理结果集和管理连接。这不仅繁琐, 而且容易出错 [p.5]
- ▷ **ORM 的四大目标** [p.5]:
 - **简化 JDBC 操作**: 封装重复的连接管理和异常处理逻辑, 提供更友好的 API
 - **提高生产力 (Productivity)**: 将关系数据以对象的形式表示, 便于开发者理解与操作
 - **跨数据库移植性 (Portability)**: 通过方言 (Dialect) 抽象提供一定程度的数据库无关性
 - **提供高级功能**: 缓存 (Cache)、延迟加载 (Lazy Loading)、事务管理 (Transaction)、批量操作 (Batch Processing) 等

1.3 JDBC API 回顾 [p.6]

- ▷ **JDBC (Java Database Connectivity)**: Java 标准库提供的 API, 用于执行 SQL 语句并处理结果集 [p.6]
- ▷ **核心接口体系** [p.6]:
 - **Driver**: 数据库驱动, 负责建立与数据库的连接
 - **Connection**: 数据库连接对象, 通过 `DriverManager.getConnection()` 获取
 - **Statement**: 用于执行静态 SQL 语句 (无参数)
 - **PreparedStatement**: 预编译 SQL 语句, 支持参数绑定, 防止 SQL 注入
 - **ResultSet**: 查询结果集, 通过游标遍历每一行数据
- ▷ **JDBC 编程的典型流程** [p.6]:
 - 加载驱动 → 获取连接 → 创建 Statement/PreparedStatement → 执行 SQL → 遍历 ResultSet → 映射到 Java 对象 → 关闭资源 (逆序)

▷ **JDBC 编程的四个痛点** [p.6]:

- 大量样板代码 (Boilerplate Code): 每个操作都要写完整的连接-执行-释放流程
- 代码冗长, 容易出错
- 一旦忘记释放资源可能导致连接泄漏 (Connection Leak), 资源耗尽
- SQL 硬编码在 Java 代码中, 维护和修改不便

1.4 示例数据库——userinfo 表

[p.8]

- ▷ 课程演示使用的示例数据库: 某网站用户信息表 `userinfo` [p.8]
- ▷ 表结构字段: `id` (INT, 主键)、`name` (VARCHAR)、`email` (VARCHAR)、`age` (INT) [p.8]
- ▷ 该表将贯穿整个章节, 作为 JDBC / JdbcTemplate / MyBatis / Hibernate / JPA 统一演示的基础 [p.8]

1.5 传统 JDBC 编程代码范例

[p.9–11]

▷ **DBUtil 工具类** [p.9]:

- 静态配置:`driver = "com.mysql.jdbc.Driver"`、`url = "jdbc:mysql://127.0.0.1:3306/demo"`、`username = "root"`、`password = ""`
- `getConnection()`: 通过 `Class.forName(driver)` 加载驱动, `DriverManager.getConnection()` 返回连接
- `closeConnection(Connection) / closeStatement(Statement) / closePreparedStatement(PreparedStatement) / closeResultSet(ResultSet)`: 资源释放方法族

▷ **INSERT 操作 (insertUser)** [p.10]:

- 获取 Connection → 预编译 INSERT SQL (? 占位符) → `setInt(1, id)` / `setString(2, name)` / `setString(3, email)` / `setInt(4, age)` 设置参数 → `executeUpdate()` → 关闭 `PreparedStatement`、`Connection`
- 返回值为受影响的行数 (1 表示成功)

▷ **SELECT 操作 (selectAllUser)** [p.11]:

- 获取 Connection → 创建 Statement → `executeQuery("SELECT * FROM userinfo")` → `while(resultSet.next())` 循环 → 逐字段 `getInt(1)` / `getString(2)` / `getString(3)` / `getInt(4)` 映射到 User 对象 → 加入 `ArrayList` → 关闭 `ResultSet`、`Statement`、`Connection`
- 注意: 字段编号从 1 开始 (非 0)

1.6 ORM 的四种实现方式 (演进层次)

[p.7]

- ▷ **Level 1 裸 JDBC**: 最底层, 完全手动控制, 代码量大且易出错, 连接资源管理繁琐 [p.7]
- ▷ **Level 2 简化的 JDBC**: 如 Spring JDBC Template, 封装连接管理和异常处理, 但 SQL 仍需手写 [p.7]
- ▷ **Level 3 一半自动 ORM 框架**: 如 MyBatis, 开发者编写 SQL, 框架负责参数映射和结果对象映射 [p.7]
- ▷ **Level 4 全自动 ORM 框架**: 如 Hibernate、Spring Data JPA, 框架自动生成 SQL, 开发者只需操作对象 [p.7]
- ▷ 四种方式的抽象层次由低到高, 开发效率递增, 但对 SQL 的精细控制力递减 [p.7]

2 Spring JDBC 模板 (JdbcTemplate)

2.1 JdbcTemplate 简介与架构

[p.13–14]

- ▷ **JdbcTemplate**: Spring 框架在 JDBC 层提供的中心委托类 (Central Delegate Class), 属于 JDBC 核心包中的核心组件 [p.13]
- ▷ **设计思想**: 采用模板方法模式 (Template Method Pattern), 封装典型的 JDBC 流程: 获取连接 → 创建语句 → 执行 SQL → 遍历结果集 → 处理异常 → 关闭资源, 让开发者只需专注于 SQL 编写和结果映射

[p.13]

- ▷ **架构位置**：位于程序员代码和 JDBC API 之间的抽象层，提供统一的模板方法，在保留代码灵活性的基础上尽量减少持久化代码 [p.14]
- ▷ **三大回调接口 (Callback Interfaces)** [p.13]:
 - **PreparedStatementCreator**：定义 SQL 创建逻辑，负责生成 PreparedStatement
 - **ResultSetExtractor<T>**：提取整个 ResultSet 并返回对象，适合复杂映射
 - **RowMapper<T>**：将 ResultSet 的单行数据映射为 Java 对象
- ▷ **其他核心特性** [p.13]:
 - **统一异常处理**：将受检异常 SQLException 转换为 Spring 的非受检异常 DataAccessException 层次结构（如 DataAccessException、DataIntegrityViolationException 等）
 - **线程安全 (Thread-safe)**：配置完成后可在多线程场景安全共享，无需加锁
 - **日志记录**：所有 SQL 操作在 DEBUG 级别记录，便于排查问题

2.2 JdbcTemplate 使用步骤

[p.15–16]

- ▷ **Step 1 —引入依赖** [p.15]:
 - **spring-boot-starter-jdbc**：Spring Boot JDBC 启动器
 - **数据库驱动**：如 **mysql-connector-java** (MySQL)、**postgresql** (PostgreSQL) 等
- ▷ **Step 2 —配置数据源 (DataSource)** [p.15]:
 - 在 **application.yml / application.properties** 中配置数据库连接信息
 - 典型配置项：**spring.datasource.url**、**spring.datasource.username**、**spring.datasource.password**、**spring.datasource.driver-class-name**
- ▷ **Step 3 —注入 JdbcTemplate 并使用 API** [p.16]:
 - 通过 **@Autowired** 注入 JdbcTemplate 对象
 - 常用 API:
 - * **query(sql, rowMapper, args...)**：查询列表，返回 List<T>
 - * **queryForObject(sql, rowMapper, args...)**：查询单个对象
 - * **queryForList(sql, args...)**：查询返回 List<Map<String, Object>>
 - * **update(sql, args...)**：执行 INSERT/UPDATE/DELETE
 - * **batchUpdate(sql, batchArgs...)**：批量操作
 - * **execute(sql)**：执行任意 SQL

2.3 JdbcTemplate 的优缺点分析

[p.17]

- ▷ **优点** [p.17]:
 - 简单、轻量，零学习成本（熟悉 JDBC 即可上手）
 - 适用于小型项目、简单查询场景
 - 对 SQL 有完全控制需求时非常合适
 - Spring 原生支持，与 Spring 生态无缝集成
- ▷ **缺点** [p.17]:
 - SQL 仍需手动编写，且在 Java 代码中拼写（无 XML/注解分离，SQL 与 Java 代码耦合）
 - 结果映射需手动指定 **RowMapper**，字段多时代码量大
 - **没有缓存机制**，每次查询都直接访问数据库
 - 关联查询（多表 JOIN）复杂时，结果映射非常麻烦
- ▷ **适用边界**：当查询变得复杂、需要动态拼接 SQL 语句或涉及多表关联映射时，应当考虑使用更高级的 ORM

框架 [p.17]

3 MyBatis 框架详解

3.1 MyBatis 简介与历史

[p.19–20]

▷ 历史沿革 [p.19]:

- 原名 **iBatis**，由 Clinton Begin 于 2002 年发布
- 2010 年项目被贡献给 Google Code，更名为 **MyBatis**
- 后迁移至 GitHub，目前最新版本为 MyBatis 3.x

▷ 官网: <https://mybatis.org/mybatis-3/> [p.20]

▷ 定位: 一款优秀的持久层框架 (**Persistence Framework**), 支持自定义 SQL、存储过程 (Stored Procedure) 以及高级映射 (Advanced Mapping) [p.19]

▷ MyBatis 免除了几乎所有的 JDBC 代码以及设置参数和获取结果集的工作 [p.19]

▷ 可以通过简单的 **XML 或注解 (Annotation)** 来配置和映射原始类型、接口和 Java POJO (Plain Old Java Objects) 为数据库中的记录 [p.19]

3.2 MyBatis 核心特点 (七大特性)

[p.22]

▷ (1) 自定义 SQL: 开发者编写完整的 SQL 语句, 框架不会生成或修改 SQL, 保证对 SQL 的精确控制; 适合需要 DBA 审核 SQL 或复杂优化查询的场景 [p.22]

▷ (2) 参数映射与结果映射 [p.22]:

- 参数映射: 将 Java 对象的属性自动映射为 SQL 预编译语句参数, 使用 `#{}` 占位符 (安全的预编译绑定) 或 `${}` (字符串替换, 有 SQL 注入风险, 仅用于动态表名/列名)
- 结果映射: 根据配置 (`resultType` 或 `resultMap`) 自动将结果集的每一行映射为 Java 对象
- `<resultMap>` 支持复杂的映射配置: `<id>` (主键)、`<result>` (普通字段)、`<association>` (一对一关联)、`<collection>` (一对多关联)

▷ (3) 动态 SQL (Dynamic SQL) [p.22]: 提供 `<if>`、`<choose>`、`<when>`、`<otherwise>`、`<foreach>`、`<where>`、`<set>`、`<trim>` 等标签在 XML 中动态拼接 SQL 条件, 避免在 Java 代码中进行字符串拼接, 提高可读性和可维护性

▷ (4) 映射文件结构简洁 [p.22]: 一个 Mapper XML 文件包含:

- 根元素 `<mapper namespace="...">` 对应 Mapper 接口全限定名
- SQL 元素: `<select>`、`<insert>`、`<update>`、`<delete>`
- `id` 对应接口方法名, `parameterType` 指定入参类型, `resultType/resultMap` 指定返回类型
- `<sql>` 标签定义可重用的 SQL 片段 (通过 `<include refid="...">` 引用)

▷ (5) 缓存机制 (Cache) [p.22]:

- 一级缓存 (Local Cache / L1 Cache): SqlSession 级别, 默认开启, 同一 SqlSession 内执行相同的 SELECT 语句直接从缓存返回; 执行 INSERT/UPDATE/DELETE 提交后缓存自动清空
- 二级缓存 (Second-level Cache / L2 Cache): Mapper 级别 (namespace 级别), 需在 Mapper XML 中配置 `<cache/>` 标签手动开启; 跨 SqlSession 共享, 支持集成第三方缓存 (Ehcache、Redis、Hazelcast 等)
- 查询优先级: 二级缓存 → 一级缓存 → 数据库

▷ (6) 插件机制 (Plugin / Interceptor) [p.22]:

- 基于 Java 动态代理 (Dynamic Proxy) + 责任链模式 (Chain of Responsibility) 实现
- 可拦截的四大对象和方法:
 - * **Executor**: `update()`、`query()`、`flushStatements()`、`commit()`、`rollback()`

- * `StatementHandler`: `prepare()`、`parameterize()`、`batch()`、`update()`、`query()`
- * `ParameterHandler`: `getParameterObject()`、`setParameters()`
- * `ResultSetHandler`: `handleResultSets()`、`handleOutputParameters()`
- 典型应用: 分页插件 (`PageHelper`)、性能监控分析、SQL 日志记录、读写分离、SQL 阻断/审计
- ▷ (7) 与 Spring 集成良好 [p.22]: 通过 `mybatis-spring-boot-starter`, Spring Boot 可以自动扫描 Mapper 接口 (`@MapperScan` 或 `@Mapper` 注解) 并创建代理对象, 与 Spring 的声明式事务管理 (`@Transactional`) 完美结合

3.3 MyBatis 基本使用流程

[p.23–26]

Step 1. 引入依赖 [p.23]:

- ▷ `mybatis-spring-boot-starter`: MyBatis 与 Spring Boot 集成启动器
- ▷ 数据库驱动: 如 `mysql-connector-java`

Step 2. 配置框架 [p.23]: 在 `application.yml` 中配置:

- ▷ 数据源: `spring.datasource.url` / `username` / `password` / `driver-class-name`
- ▷ MyBatis 配置: `mybatis.mapper-locations` (指定 Mapper XML 文件路径, 如 `classpath:mapper/*.xml`)、`mybatis.type-aliases-package` (指定实体类包路径)、`mybatis.configuration.map-underscore-to-camel-case` (开启驼峰命名自动映射)

Step 3. 创建实体类 (Entity / POJO) [p.24]: 创建与数据库表字段对应的 Java 类 (如 `User` 类, 属性 `id`、`name`、`email`、`age`), 属性名建议与字段名对应 (或通过 `map-underscore-to-camel-case` 自动转换) [p.24]

Step 4. 创建 Mapper 接口 [p.24]:

- ▷ 使用 `@Mapper` 注解或 `@MapperScan("com.xxx.mapper")` 配置类扫描
- ▷ 定义数据访问方法签名, 如: `User selectById(int id)`、`int insertUser(User user)`、`List<User> selectAll()`

Step 5. 编写 Mapper XML 映射文件 [p.25]:

- ▷ `<mapper namespace="com.xxx.mapper.UserMapper">`
- ▷ `<select id="selectById" parameterType="int" resultType="com.xxx.entity.User"> SELECT * FROM userinfo WHERE id = #{id} </select>`
- ▷ `<insert id="insertUser" parameterType="com.xxx.entity.User"> INSERT INTO userinfo VALUES (#{id}, #{name}, #{email}, #{age}) </insert>`
- ▷ `<select id="selectAll" resultType="com.xxx.entity.User"> SELECT * FROM userinfo </select>`

Step 6. 在 Service 层注入并使用 Mapper [p.26]:

- ▷ 通过 `@Autowired` 或 `@Resource` 注入 Mapper 接口 (MyBatis-Spring 自动生成代理实现对象)
- ▷ 在 Service 方法中直接调用 Mapper 方法, 配合 `@Transactional` 进行事务管理

3.4 动态 SQL 详细说明

[p.22, p.27]

- ▷ 设计目的: 根据运行时条件动态拼接 SQL 语句, 避免在 Java 代码中使用大量 `if-else` 进行字符串拼接, 保持 SQL 可读性和可维护性 [p.22, p.27]
- ▷ 常用动态 SQL 标签及其用法:
 - `<if test="condition">`: 条件判断, 当 `test` 中表达式为 `true` 时包含标签内的 SQL 片段; 例如 `<if test="name != null and name != ''"> AND name = #{name} </if>`
 - `<choose>` / `<when>` / `<otherwise>`: 多条件分支选择, 类似 Java 的 `switch-case-default` 结构; `<when>` 条件成立则采用, 都不成立则采用 `<otherwise>`

- **<where>**: 智能处理 WHERE 子句, 自动去除多余的 AND/OR 前缀; 若标签内无内容则不输出 WHERE 关键字
- **<set>**: 动态 UPDATE 中的 SET 子句, 智能去除尾部逗号; 若标签内无内容则不输出 SET 关键字
- **<foreach>**: 遍历集合, 常用于 IN 查询:
 - * 属性: **item** (当前元素)、**index** (索引)、**collection** (集合名)、**open** (起始字符串)、**close** (结束字符串)、**separator** (分隔符)
 - * 示例: `<foreach collection="ids" item="id" open="(" close=")" separator=","> #{id} </foreach>`
- **<trim>**: 更灵活的空白/前缀/后缀处理, 可替代 **<where>** 和 **<set>**:
 - * **prefix**: 添加前缀; **suffix**: 添加后缀
 - * **prefixOverrides**: 去除多余前缀 (如 AND | OR); **suffixOverrides**: 去除多余后缀 (如逗号)
- **<sql> + <include>**: 定义 SQL 片段并在多个语句中复用, 减少重复代码
- ▷ **动态 SQL 示例——条件查询** [p.27]:
 - 根据 name、email、age 三个可选条件动态查询 userinfo 表
 - 使用 **<where>** 包裹多个 **<if>** 标签, 自动处理 WHERE 关键字和 AND 前缀

3.5 MyBatis 注解方式 (Annotation-based)

[p.19, p.22]

- ▷ 除了 XML 映射, MyBatis 也支持通过注解直接在 Mapper 接口上定义 SQL [p.19]:
 - `@Select("SELECT * FROM userinfo WHERE id = #{id}")`
 - `@Insert("INSERT INTO userinfo VALUES(#{id}, #{name}, #{email}, #{age})")`
 - `@Update("UPDATE userinfo SET name=#{name} WHERE id=#{id}")`
 - `@Delete("DELETE FROM userinfo WHERE id=#{id}")`
 - `@Results / @Result`: 定义结果映射 (替代 **<resultMap>**)
 - `@SelectProvider / @InsertProvider` 等: 引用动态 SQL 提供类
- ▷ 注解方式适用于简单 SQL 场景, 复杂动态 SQL 仍推荐使用 XML 方式 [p.22]

4 Hibernate 框架

4.1 Hibernate 简介

[p.29, p.31]

- ▷ Hibernate 是 Java 生态中全自动的 **ORM 解决方案**, 提供自然且类型安全的方式访问关系数据 [p.29]
- ▷ 能快速编写复杂查询和处理结果, 也可以轻松将内存中的对象修改同步到数据库 [p.29]
- ▷ 遵循事务的 ACID 属性 (Atomicity, Consistency, Isolation, Durability), 提供调优性能的能力 [p.29]
- ▷ Hibernate 与 **Jakarta Persistence (JPA)** 兼容, 可在支持 JPA 的任何环境中使用 [p.29]
- ▷ 官网: <https://hibernate.org/> [p.31]

4.2 JPA 规范 (Java Persistence API)

[p.30]

- ▷ **JPA (Java Persistence API)**: Java EE / Jakarta EE 的众多规范之一, 即 ORM 规范, 规定了对象持久化的标准 API [p.30]
- ▷ **关键认知**: JPA 本身并不提供实现, 只提供规范接口; 广义上来说 JDBC、JdbcTemplate、MyBatis、Hibernate 都可以认为是 JPA 规范的具体实现 [p.30]
- ▷ **JPA 规范的三大组成部分** [p.30]:
 - **ORM 映射元数据 (Mapping Metadata)**: 定义实体类与数据库表的映射关系, 支持 XML 和注解 (Annotation) 两种配置方式
 - **Java 调用的 API 接口**: 包括 **EntityManager** (实体管理器)、**EntityTransaction** (事务)、**Query / TypedQuery** (查询) 等核心接口

- 面向对象的查询语言 JPQL (Java Persistence Query Language): 类似 SQL 但操作的是实体对象而非数据库表, 在编译时进行类型检查
- ▷ 主流 JPA 实现: Hibernate、EclipseLink、OpenJPA 等 [p.30]

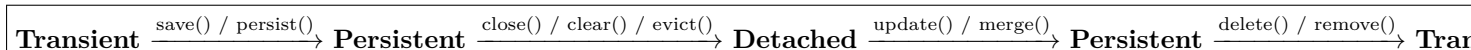
4.3 Hibernate 核心特点 (六大特性) [p.32]

- ▷ (1) 完全的 ORM 能力 [p.32]:
 - 支持基于注解 (`@Entity`、`@Table`、`@Column` 等) 或 XML 定义实体与表的映射
 - 支持继承映射策略 (单表继承 `SINGLE_TABLE`、连接表继承 `JOINED`、每个类一张表 `TABLE_PER_CLASS`)
 - 支持关联关系映射: `@ManyToOne` (多对一)、`@OneToMany` (一对多)、`@ManyToMany` (多对多)、`@OneToOne` (一对一)
- ▷ (2) 查询语言 HQL (Hibernate Query Language) [p.32]:
 - 面向对象的查询语言, 操作的是实体类和属性而非表和列
 - 支持大部分 ANSI SQL 功能 (聚合、分组、子查询、连接等)
 - 在编译时对查询进行类型检查, 减少运行时错误
 - 也支持原生 SQL (Native SQL) 和 Criteria API (类型安全的编程式查询)
- ▷ (3) 缓存机制 [p.32]:
 - 一级缓存: Session 级别, 默认开启, 与 Session 生命周期绑定
 - 二级缓存: SessionFactory 级别, 可插拔设计, 支持自定义缓存提供者 (Ehcache、Infinispan、Hazelcast、Redis 等)
 - 查询缓存 (Query Cache): 缓存 HQL/Criteria 查询的结果集标识
- ▷ (4) 延迟加载与级联操作 [p.32]:
 - 延迟加载 (Lazy Loading): 通过 `FetchType.LAZY` 配置, 实体关联可以按需加载, 避免一次拉取所有关联对象; 需注意 N+1 查询问题
 - 级联操作 (Cascade): 级联持久化 (`CascadeType.PERSIST`)、级联合并 (`MERGE`)、级联删除 (`REMOVE`)、级联刷新 (`REFRESH`)、全级联 (`ALL`)
 - 级联操作方便维护对象之间的关联关系, 但需谨慎使用以免性能问题
- ▷ (5) JPA 标准支持 [p.32]:
 - Hibernate 是 JPA 规范的参考实现 (Reference Implementation) 之一
 - 使用 JPA 标准注解和 API 编写的代码可以与 EclipseLink 等其他 JPA 实现兼容
 - 开发者可以在此之上使用 Spring Data JPA 等更高层抽象
- ▷ (6) 性能与扩展性 [p.32]:
 - 可调节的 SQL 生成 (通过 `Dialect` 适配不同数据库)
 - 批量更新 (Batch Processing)、批量抓取 (Batch Fetching)
 - 二级缓存优化、只读实体优化
 - 适用于高并发和复杂查询场景

4.4 Hibernate 持久化对象的三种状态 (常考!) [p.33–34]

- ▷ 这是 Hibernate / JPA 的核心概念, 描述了对对象从创建到持久化到数据库的完整生命周期 [p.33]
- ▷ (1) 瞬时状态 / 临时状态 (Transient) [p.33]:
 - 由 `new` 操作符创建, 且尚未与 Hibernate Session 关联的对象
 - 不会被持久化到数据库, 也不会被赋予持久化标识 (Identifier / Primary Key)
 - 若在程序中没有被引用, 会被垃圾回收器 (GC) 正常销毁
 - 转换为持久状态的方法: 调用 `session.save(obj)` 或 `session.persist(obj)`

- Hibernate 自动执行必要的 INSERT SQL 语句
- ▷ (2) 持久化状态 (Persistent) [p.33]:
 - 持久实例可能刚被保存 (save / persist) 或刚被加载 (get / load / find)
 - 存在于相关联的 Session 作用范围内, 受 Session 管理
 - 脏检查 (Dirty Checking): Hibernate 会自动检测处于持久状态的对象的任何属性改动 (通过快照对比机制), 在当前操作单元执行完毕时 (事务提交 / flush) 自动将对象数据与数据库同步
 - 开发者不需要手动执行 UPDATE 语句
 - 删除持久对象调用 session.delete(obj) / session.remove(obj), 同样不需要手动 DELETE
- ▷ (3) 脱管状态 / 游离状态 (Detached) [p.33]:
 - 与持久对象关联的 Session 被关闭后 (session.close()、session.clear()、session.evict(obj)), 对象变为脱管状态
 - 对脱管对象的引用依然有效, 对象可继续被修改
 - 脱管对象如果重新关联到某个新的 Session 上 (通过 session.update(obj) 或 session.merge(obj)), 会再次转变为持久状态
 - 脱管期间的改动将在重新关联后被持久化到数据库
 - update() 与 merge() 的区别: update() 要求数据库中存在对应记录 (否则抛异常), merge() 不存在则插入、存在则更新
- ▷ 状态转换图 (文字描述) [p.33–34]:



- ▷ Session 核心方法速记 [p.33]:
 - get(Class, id): 立即加载, 返回实体或 null (不存在时)
 - load(Class, id): 延迟加载, 返回代理对象, 访问属性时才查库; 不存在时抛 ObjectNotFoundException
 - save(obj) / persist(obj): 将瞬时对象变为持久
 - update(obj) / merge(obj): 将脱管对象重新变为持久
 - delete(obj) / remove(obj): 将持久对象变为瞬时 (删除数据库记录)

5 Spring Data JPA

5.1 Spring Data JPA 简介 [p.37]

- ▷ Spring Data JPA: Spring 提供的一套简化 JPA 开发的框架 [p.37]
- ▷ 核心理念: 按照约定优于配置 (Convention over Configuration), 通过方法命名规则 (Method Naming Convention) 设计 DAO 层接口, 自动生成查询实现, 无需编写 SQL 或 JPQL [p.37]
- ▷ 除了基本的 CRUD 操作之外, 还提供分页 (Pagination)、排序 (Sorting)、复杂查询 (@Query 注解、Specification 动态查询) 等功能 [p.37]
- ▷ 底层依赖 JPA 规范 (如 Hibernate 作为 JPA 实现), 是 JPA 之上的更高层抽象 [p.37]

5.2 Spring Data JPA 使用步骤 [p.38–39]

Step 1. 引入依赖 [p.38]:

- ▷ spring-boot-starter-data-jpa: Spring Data JPA 启动器 (已包含 Hibernate 和 Spring Data JPA)
- ▷ 数据库驱动: 如 mysql-connector-java

Step 2. 创建实体类 [p.38]: 使用 JPA 标准注解配置映射:

- ▷ `@Entity`: 标识该类为 JPA 实体类
- ▷ `@Table(name = "userinfo")`: 指定映射的数据库表名（不指定则默认类名）
- ▷ `@Id`: 标识主键字段
- ▷ `@GeneratedValue(strategy = GenerationType.IDENTITY)`: 主键生成策略（IDENTITY= 自增、AUTO= 自动、SEQUENCE= 序列、TABLE= 表生成器）
- ▷ `@Column(name = "field_name", nullable = false, length = 50)`: 指定列名及约束
- ▷ `@Transient`: 标识该字段不映射到数据库（非持久化字段）

Step 3. 创建 Repository 接口 [p.39]:

- ▷ 继承 `JpaRepository<T, ID>`（推荐）或 `CrudRepository<T, ID>`
- ▷ `JpaRepository<T, ID>` 相比 `CrudRepository<T, ID>` 额外提供: `findAll(Sort)`、`findAll(Pageable)`、`flush()`、`saveAndFlush()`、`deleteInBatch()` 等
- ▷ 无需提供实现类，Spring Data JPA 自动生成代理实现
- ▷ 可声明抽象方法，通过方法命名约定自动生成查询：
 - `findByName(String name)`: 按 name 精确查询
 - `findByNameLike(String pattern)`: 模糊查询
 - `findByAgeGreaterThan(int age)`: 大于查询
 - `findByAgeBetween(int start, int end)`: 范围查询
 - `findByNameAndAge(String name, int age)`: 多条件 AND
 - `findByNameOrEmail(String name, String email)`: 多条件 OR
 - `findByAgeOrderByNameDesc(int age)`: 排序
 - 支持的关键字: `And`, `Or`, `Between`, `LessThan`, `GreaterThan`, `Like`, `NotLike`, `In`, `NotIn`, `IsNull`, `IsNotNull`, `OrderBy`, `Distinct`, `IgnoreCase` 等
- ▷ 可使用 `@Query("SELECT u FROM User u WHERE u.age > ?1")` 注解编写自定义 JPQL 查询
- ▷ 可使用 `@Query(value = "SELECT * FROM userinfo WHERE age > ?1", nativeQuery = true)` 执行原生 SQL
- ▷ 可使用 `@Modifying + @Query` 执行 UPDATE/DELETE 操作

Step 4. 在 Service 中使用 [p.39]:

- ▷ 通过 `@Autowired` 注入 Repository 接口
- ▷ 直接调用继承的 CRUD 方法或自定义命名方法
- ▷ 配合 `@Transactional` 注解进行事务管理

5.3 Spring Data JPA 常用内置方法

[p.37, p.39]

- ▷ **CRUD 操作**: `save(S entity)`、`saveAll(Iterable<S> entities)`、`findById(ID id)`（返回 `Optional<T>`）、`findAll()`、`deleteById(ID id)`、`delete(T entity)`、`count()`、`existsById(ID id)`
- ▷ **分页查询**: `findAll(Pageable pageable)`，使用 `PageRequest.of(page, size, Sort)` 构造分页参数，返回 `Page<T>` 对象（包含内容列表、总页数、总记录数等）
- ▷ **排序查询**: `findAll(Sort sort)`，使用 `Sort.by("age").descending()` 构造排序参数
- ▷ **返回值类型**: `Optional<T>`（单条）、`List<T>`（多条）、`Page<T>`（分页）、`Slice<T>`（切片，无需 count）

6 MyBatis 生态插件

6.1 MyBatis “三剑客” (Three Musketeers)

[p.41]

- ▷ 围绕 MyBatis 基础功能，由第三方设计和开发的各种工具、插件的常用组合，简称“三剑客” [p.41]:
 - **MyBatis-generator**: 自动代码生成工具

- **MyBatis Plugin**: IDE 开发辅助插件
 - **MyBatis 分页 PageHelper**: 通用分页插件
- 此外还有 **MyBatis-Plus** (后文详述), 功能更为全面 [p.41]

6.2 MyBatis-generator (代码生成器) [p.42]

- **定位**: 基于 MyBatis 框架的自动代码生成工具 [p.42]
- **主要功能**: 根据数据库表结构自动生成 Java Bean (实体类)、Mapper 接口和 Mapper XML 映射文件 [p.42]
- **只需少量配置即可运行** [p.42]:
- 数据库连接信息 (JDBC URL、用户名、密码)
 - 生成策略: 表选择 (哪些表生成)、生成路径 (Java 模型/接口/XML 输出目录)
 - 可选配置: 注释生成、toString/hashCode/equals 方法生成、Example 类生成 (用于复杂条件查询)
- **适用场景**: 项目初期或数据库表结构频繁变更时, 快速生成基础 CRUD 代码, 避免手工编写重复的实体类和映射文件 [p.42]

6.3 MyBatis Plugin (IDE 插件) [p.41]

- IDE 开发辅助插件, 如 IntelliJ IDEA 的 Free MyBatis Plugin / MyBatisX [p.41]
- **主要功能**:
- Mapper 接口方法与 XML 中 SQL 语句之间一键跳转
 - SQL 语法高亮和自动补全
 - Mapper XML 中 namespace 和 id 的智能提示和验证
 - 实体类字段与 SQL 字段的对应关系展示
- 显著提升 MyBatis 开发体验和效率 [p.41]

6.4 MyBatis 分页 PageHelper [p.43]

- **分页的必要性** [p.43]:
- 查询大数据量数据库表时的必备功能
 - 减少数据库查询压力 (避免全表扫描返回大量数据)
 - 降低客户端的数据加载量, 提升用户体验
- **原生 MyBatis 分页的痛点**: 手动实现分页较为繁琐和复杂, 需要在 SQL 中拼写 LIMIT/OFFSET (各数据库语法不同), 且需额外编写 COUNT 查询 [p.43]
- **PageHelper 插件特点** [p.43]:
- 通用分页插件, 支持多种数据库 (MySQL、Oracle、PostgreSQL、SQL Server、DB2 等)
 - 较少的配置即可实现复杂的分页功能
 - **核心用法**: 在查询前调用 `PageHelper.startPage(pageNum, pageSize)` 一行代码即可实现物理分页
 - **实现原理**: 基于 MyBatis 拦截器 (Interceptor), 拦截 `Executor.query()` 方法, 自动在执行 SQL 前添加分页逻辑, 并执行 COUNT 查询获取总数
 - 返回 `PageInfo<T>` 对象, 包含分页的完整信息 (当前页、总页数、总记录数、每页大小、是否为首页/尾页等)

7 MyBatis-Plus (MyBatis 增强工具)

7.1 MyBatis-Plus 简介 [p.44]

- **官网**: <https://baomidou.com/> [p.44]
- MyBatis-Plus 是目前非常流行的基于 MyBatis 的增强工具, 荣获 “2019 年开源中国最受欢迎开发工具类软件 TOP 1” [p.44]

- ▷ 设计目标：简化开发、提高效率，让 MyBatis 使用更加便捷 [p.44]
- ▷ 核心理念 [p.44]:
 - 只做增强不做改变：完全兼容 MyBatis 原生功能，在原有基础上做增强
 - 无侵入：引入后不会对现有工程产生任何影响
 - 损耗小：启动时自动注入基本 CRUD，性能几乎无损耗
 - 直接面向对象操作：通过通用 BaseMapper 和条件构造器实现面向对象的数据库操作
- ▷ 实现原理：大部分功能基于 AOP (Aspect-Oriented Programming) 实现 [p.44]
- ▷ 功能覆盖：基本囊括“三剑客”的全部功能（代码生成、分页等），还提供很多独有特性 [p.44]

7.2 MyBatis-Plus 框架结构

[p.45]

- ▷ 在 MyBatis 基础之上进行增强，核心是对 BaseMapper<T> 的封装 [p.45]
- ▷ 内置通用 CRUD: insert(entity)、deleteById(id)、deleteByMap(map)、updateById(entity)、selectById(id)、selectList(wrapper)、selectPage(page, wrapper) 等 [p.45]
- ▷ 条件构造器 Wrapper 体系 [p.45]:
 - AbstractWrapper<T>: 抽象基类
 - QueryWrapper<T>: 查询条件构造器，通过方法链式调用构建 WHERE 条件
 - UpdateWrapper<T>: 更新条件构造器，直接设置要更新的字段及值
 - LambdaQueryWrapper<T>: Lambda 表达式风格的查询条件，类型安全，避免字段名硬编码
 - LambdaUpdateWrapper<T>: Lambda 表达式风格的更新条件
- ▷ 核心机制：通过反射 + AOP 在运行时动态生成 SQL，实现零 XML 的基本 CRUD [p.45]

7.3 MyBatis-Plus 核心特性

[p.44–45]

- ▷ 通用 CRUD：继承 BaseMapper<T> 即可获得全套 CRUD 方法，无需编写 XML [p.44]
- ▷ 条件构造器：支持链式调用，方法包括：
 - 比较：eq()（等于）、ne()（不等于）、gt()（大于）、ge()（大于等于）、lt()（小于）、le()（小于等于）
 - 模糊：like()、notLike()、likeLeft()、likeRight()
 - 范围：between()、notBetween()、in()、notIn()
 - 空值：isNull()、isNotNull()
 - 分组/排序：groupBy()、orderByAsc()、orderByDesc()、having()
 - 逻辑：and()、or()、exists()、notExists()
- ▷ Lambda 表达式查询: LambdaQueryWrapper<User> wrapper = new LambdaQueryWrapper<>(); wrapper.eq(User::getName, "张三").gt(User::getAge, 18); [p.44]
- ▷ ActiveRecord 模式：实体类继承 Model<T> 后，可直接通过实体对象调用 CRUD 方法，如 user.insert()、user.updateById() [p.44]
- ▷ 主键策略：内置多种主键生成策略，支持雪花算法 (Snowflake)、UUID、数据库自增等 [p.44]
- ▷ 自动填充：通过 @TableField(fill = FieldFill.INSERT) 实现创建时间、更新时间等字段的自动填充 [p.44]
- ▷ 其他企业级功能：逻辑删除 (@TableLogic)、乐观锁 (@Version)、多租户、SQL 注入器、自动分页、代码生成器等 [p.44]

7.4 MyBatis-Plus 使用步骤

[p.46–48]

Step 1. 引入依赖 [p.46]:

- ▷ mybatis-plus-boot-starter: 替代原有的 mybatis-spring-boot-starter
- ▷ 数据库驱动：如 mysql-connector-java

Step 2. 配置框架 [p.46]: 在 application.yml 中配置与 MyBatis 类似，但使用 MyBatis-Plus 专有配置项前

缀，如 `mybatis-plus.mapper-locations` 等

Step 3. 创建实体类 [p.47]: 使用 MyBatis-Plus 专用注解:

- `@TableName("table_name")`: 指定对应的数据库表名; 可配置全局表前缀 (如 `t_`) 自动拼接
- `@TableId(type = IdType.ASSIGN_ID)`: 指定主键及生成策略 (默认雪花算法, `ASSIGN_ID`)
- `@TableField("column_name")`: 指定属性与数据库列名的映射
- `@TableField(exist = false)`: 标识该字段在数据库中不存在
- `@TableLogic`: 逻辑删除字段 (值为 0 表示未删除, 1 表示已删除)
- `@Version`: 乐观锁版本号字段

Step 4. 创建 Mapper 接口 [p.47]:

- 继承 `BaseMapper<User>` 即可获得内置通用 CRUD 方法
- 无需编写对应的 Mapper XML 文件 (除非有自定义复杂 SQL)

Step 5. 使用条件构造器 Wrapper 进行复杂查询 [p.48]:

- `QueryWrapper<User> wrapper = new QueryWrapper<>(); wrapper.eq("name", " 张三").gt("age", 18).orderByDesc("id");`
- Lambda 方式: `LambdaQueryWrapper<User> lambdaWrapper = new LambdaQueryWrapper<>(); lambdaWr " 张").between(User::getAge, 20, 30);`
- `selectPage(page, wrapper)` 配合分页插件实现分页查询

8 ORM 框架对比与项目选型

8.1 JDBC 到 ORM 的完整演进路径

[p.6–48]

- **原生 JDBC** [p.6, p.9–11]: 最底层, 完全手动控制, 代码量大, 易出错, 连接管理繁琐; 优点是完全自由, 零额外依赖
- **Spring JdbcTemplate** [p.13–17]: 封装连接管理和异常处理, 但仍需手动编写 SQL 和 `RowMapper`, 无缓存机制; 优点是简单轻量
- **MyBatis** [p.19–27]: 半自动 ORM, 开发者编写 SQL + XML/注解映射, 灵活性强, 支持动态 SQL、两级缓存、插件机制; 适合需要精细控制 SQL 的场景
- **Hibernate** [p.29–34]: 全自动 ORM, 完全的对象-关系映射, 支持 HQL 和 JPA, 自动脏检查和延迟加载, 三级缓存; 适合标准 CRUD 和复杂对象模型
- **Spring Data JPA** [p.37–39]: 基于 JPA 规范的高层抽象, 按方法命名约定自动生成查询, 分页和排序内置, 代码量最少
- **MyBatis-Plus** [p.44–48]: MyBatis 的增强工具, 通用 CRUD + 条件构造器 + 代码生成器, 兼具 MyBatis 灵活性和极高开发效率

8.2 选型讨论: 应不应该使用 ORM 框架?

[p.49]

- **是否使用** 取决于项目规模、团队能力、性能要求 [p.49]:
 - **小型项目 / 简单查询**: `JdbcTemplate` 足够, 避免过度设计; 直接使用原生 JDBC 或 `JdbcTemplate`, 代码量可控
 - **中大型项目**: ORM 框架显著提升开发效率和代码可维护性, 减少样板代码, 降低出错概率
 - **高性能要求场景**: 可能需要在 ORM 基础上做针对性优化 (如 MyBatis 手写优化 SQL), 甚至部分场景回归原生 JDBC
 - **团队技能考量**: 团队是否熟悉 ORM 框架? 学习曲线成本是否可接受?

8.3 选型讨论：如何选择具体的 ORM 框架？

[p.49]

- ▷ 选择 MyBatis / MyBatis-Plus [p.49]:
- 需要对 SQL 有完全控制权（如 DBA 要求审核所有 SQL）

– 查询复杂多变、多表关联、报表统计等场景

– 遗留数据库，表结构不规范，需要灵活映射

– 团队对 SQL 熟悉，希望直接优化查询性能

– 推荐优先使用 **MyBatis-Plus**：兼具 MyBatis 灵活性和开发效率
- ▷ 选择 Spring Data JPA / Hibernate [p.49]:
- 标准 CRUD 为主，业务逻辑集中在 Service 层

– 追求开发速度和代码简洁性

– 复杂对象模型（继承、多态、深度关联关系）

– 需要跨数据库移植性

– 团队熟悉 JPA 规范和面向对象编程
- ▷ 混合使用策略 [p.49]:
- 在一个项目中可以同时使用多种 ORM 方案

– 例如：核心业务使用 Spring Data JPA，复杂报表使用 MyBatis 手写 SQL

– 例如：MyBatis-Plus 处理基本 CRUD，复杂查询使用原生 MyBatis XML

框架核心特性对比速查表

特性	Spring JdbcTemplate	MyBatis	Hibernate / JPA	MyBatis-Plus
ORM 级别	简化 JDBC（非 ORM） [p.13]	半自动 ORM [p.19]	全自动 ORM [p.29]	半自动 ORM + 增强 [p.44]
SQL 控制	完全手动，Java 代码拼写 [p.17]	手写 SQL，XML / 注解分离 [p.22]	框架自动生成 (HQL / JPQL) [p.32]	通用 CRUD 自动 + 自定义 SQL [p.44]
参数映射	手动 RowMapper [p.17]	#{} 占位符 [p.22]	自动映射 [p.32]	自动映射 [p.44]
结果映射	RowMapper 接口实现 [p.16]	resultType / resultMap / association / collection [p.22]	@Entity + @Column 注解自动映射 [p.32, p.38]	@TableName + @TableField [p.47]
动态 SQL	无原生支持，需手动 if-else 拼接 [p.17]	标签化：<if>，<foreach>，<where>，<set>，<trim> 等 [p.22, p.27]	Criteria API / JPQL 动态拼接 [p.32]	条件构造器 Wrapper 链式调用 [p.48]
缓存机制	无 [p.17]	一级 (SqlSession) + 可选二级 (Mapper) [p.22]	一级 (Session) + 二级 (SessionFactory) + 查询缓存 [p.32]	继承 MyBatis 缓存 [p.44]
关联映射	手动逐字段组装 [p.17]	<association> / <collection> 配置 [p.22]	@ManyToOne / @OneToMany / @ManyToMany 自动映射 [p.32]	继承 MyBatis + 分页插件优化 [p.44]

特性	Spring JdbcTemplate	MyBatis	Hibernate / JPA	MyBatis-Plus
延迟加载	不支持 [p.17]	支持 (association / collection 的 fetchType) [p.22]	FetchType.LAZY 支持, 注意 N+1 问题 [p.32]	继承 MyBatis [p.44]
脏检查 (Dirty Checking)	无 [p.17]	无 [p.22]	自动检测 (快照比对), 自动 UPDATE [p.33]	无 (需显式 updateById) [p.44]
代码生成	无	MyBatis-generator [p.42]	IDE 工具支持	内置代码生成器 [p.44]
分页支持	手动 SQL LIMIT [p.17]	PageHelper 插件 [p.43]	Pageable 接口, PageRequest [p.37]	内置分页插件 + Page<T> [p.44]
事务管理	需手动管理 [p.6]	集成 Spring @Transactional [p.22]	集成 Spring @Transactional [p.32]	集成 Spring @Transactional [p.44]
学习曲线	低 [p.13]	中等 [p.22]	较高 (需理解 Session/JPA/对象状态) [p.33]	低-中等 [p.44]
适用场景	小型项目、简单查询 [p.17]	SQL 控制需求高、复杂查询 [p.22, p.49]	标准 CRUD、复杂对象模型 [p.32, p.49]	兼顾 MyBatis 灵活性和开发效率 [p.44, p.49]
代码量 (同功能)	最多	中等	少	最少 (通用 CRUD) [p.44]

本章小结与考点梳理

本章从 ORM 框架的基本概念出发, 系统地介绍了 JDBC 编程回顾、Spring JdbcTemplate、MyBatis、Hibernate/JPA、Spring Data JPA 以及 MyBatis-Plus 等主流持久化技术 [p.3, p.50]:

一级考点 (必须掌握)

- **ORM 定义与四大目标** [p.4-5]: 对象关系映射的概念、为什么需要 ORM、ORM 的四个主要目标
- **ORM 实现的四种方式** [p.7]: 从裸 JDBC 到全自动 ORM 的演进层次及各层特点
- **JDBC 核心接口** [p.6]: Driver、Connection、Statement、PreparedStatement、ResultSet 及其作用和关系
- **传统 JDBC 编程流程** [p.9-11]: DBUtil 工具类模式、INSERT/SELECT 操作的完整代码流程
- **MyBatis 七大核心特点** [p.22]: 自定义 SQL、参数/结果映射、动态 SQL、缓存、插件、Spring 集成; 每个特点需能简述
- **Hibernate 持久化对象三状态** [p.33]: Transient / Persistent / Detached 的定义、特征及相互转换方式 (常考状态转换图)
- **JPA 规范三大组成部分** [p.30]: 映射元数据、API 接口、JPQL 查询语言

二级考点 (需理解)

- **JdbcTemplate 回调接口** [p.13]: PreparedStatementCreator、ResultSetExtractor、RowMapper 各自的作用
- **JdbcTemplate 优缺点** [p.17]: 何时使用、何时需要升级到更高级 ORM
- **MyBatis 动态 SQL 标签** [p.22, p.27]: <if>、<choose>、<foreach>、<where>、<set>、<trim> 的用途和典型场景
- **MyBatis 一级缓存 vs 二级缓存** [p.22]: 作用范围、默认状态、清空条件、集成方式

- **MyBatis 插件机制原理** [p.22]: 动态代理 + 责任链, 可拦截的四大对象
- **MyBatis #{} vs \${}** [p.22]: 前者预编译安全绑定, 后者字符串替换有 SQL 注入风险
- **Hibernate 脏检查机制** [p.33]: 持久状态对象的改动如何自动同步到数据库
- **Hibernate Session 核心方法** [p.33]: get vs load、save vs persist、update vs merge 的区别
- **Spring Data JPA 方法命名规则** [p.39]: 命名关键字 (And/Or/Between/Like/GreaterThan 等) 与自动查询生成
- **MyBatis-Plus BaseMapper** [p.47]: 继承即可获得通用 CRUD, 无需编写 XML

三级考点 (了解即可)

- **MyBatis 历史沿革**: iBatis (2002) → MyBatis (2010) [p.19]
- **MyBatis “三剑客” 组成** [p.41]: generator、Plugin、PageHelper
- **MyBatis-generator 的作用** [p.42]: 根据表结构自动生成实体类和映射文件
- **PageHelper 核心用法** [p.43]: `PageHelper.startPage(pageNum, pageSize)` 一行分页
- **MyBatis-Plus 条件构造器 Wrapper 体系** [p.45, p.48]: `QueryWrapper` / `LambdaQueryWrapper` 的链式方法
- **MyBatis-Plus ActiveRecord 模式** [p.44]: 实体类继承 `Model<T>` 后直接调用 CRUD
- **框架选型原则** [p.49]: SQL 控制优先选 MyBatis, 开发效率优先选 JPA / MyBatis-Plus

框架选择口诀:

SQL 灵活用 MyBatis, 快速开发选 JPA,
两者兼得 MyBatis-Plus, 简单场景 JdbcTemplate。
三状态切换需牢记, 脏检查自动更新不用愁。